02 Chisel 与工具链

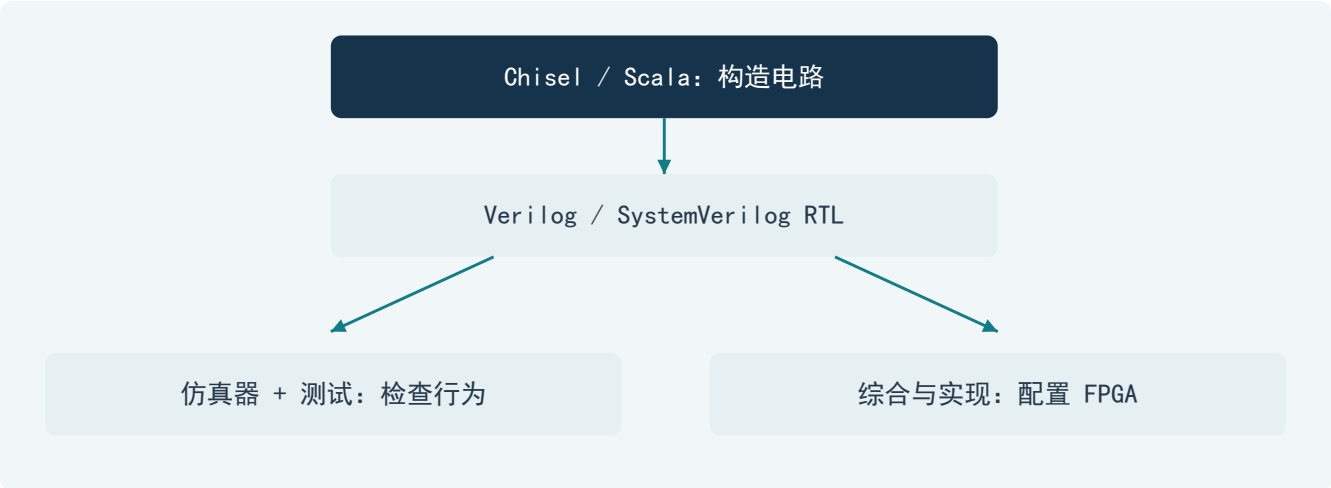
先把工具名字分清，再开始安装

会议前面写了 Trisim、Trisol，后面明确提到 Chisel 和 Chipyard。本资料按 Chisel 准备入门学习，这一名称推断仍待导师核实。

名称	是什么	你本周需要理解什么
RTL	一种硬件描述层次	状态如何在寄存器间流动
Verilog / SystemVerilog	硬件描述语言	描述接口、逻辑与寄存器
Chisel	基于 Scala 的硬件构造语言	用程序构造电路，生成 RTL
Scala	一种编程语言	读懂本册用到的少量语法
Icarus / Verilator	仿真相关工具	在计算机里检查 RTL 行为
Chipyard	集成多种生成器和工具的 SoC 研究框架	知道它在后续系统实验中的位置
FPGA	可配置逻辑硬件	后期把设计实现到真实器件上

SoC 是 System on Chip，片上系统，可包含处理器、内存接口、总线和外设。FPGA 可以实现这样的数字设计，但生成 RTL 之后仍需要综合、布局布线与板级验证。

最小工具链



Chisel 不是把任意 Scala 算法自动变成硬件。你要使用 Chisel 提供的构件，明确描述寄存器、连接与计算关系。[C1][C5]

常量、连接、比较等语法可查本册后半部分 C1-C7；先读小例子，再按需查阅。

2.1 只学够用的 Scala

六种写法就能读入门模块

写法	通俗解释	小例子
val	给一个值起名，名字不能重新指向别的值	val width = 4
class	定义一种类型；本册用来定义模块	class Gate extends Module
extends	基于已有类型扩展	extends Module
new	创建一个对象	new Bundle
{ }	把一组内容放在一起	模块、接口或条件块
import	使用库中的名字	import chisel3._

容易困惑的两种赋值

```scala
val limit = 4
val stopped = RegInit(false.B)
stopped := true.B
```

第一行创建一个 Scala 名字 limit，值为 4。第二行在构造电路时创建一个寄存器，并用 stopped 指向它。第三行使用 Chisel 的连接运算符 :=，描述这个硬件寄存器的更新输入。

所以 val stopped 并不表示“寄存器以后永远不能改变”。不可重新绑定的是 Scala 名字，硬件信号的值仍然可以随电路运行而变化。

Scala 数值与硬件数值

4 是 Scala 里的整数，可用于决定电路宽度。4.U 表示无符号硬件常量。4.W 表示位宽为 4。false.B 表示硬件布尔常量假。

```scala
val count = RegInit(0.U(4.W))
```

从里向外读：创建值为 0、宽度为 4 的无符号硬件常量，再创建以它为复位值的寄存器。

练习：val width = 8 与 val x = Wire(UInt(8.W)) 分别决定“构造电路的参数”和“电路中的信号”中的哪一个？

2.2 第一个 Chisel 组合模块

还是同一个二选一选择器

```scala
import chisel3._

class Mux2 extends Module {
  val io = IO(new Bundle {
    val a = Input(Bool())
    val b = Input(Bool())
    val sel = Input(Bool())
    val y = Output(Bool())
  })
  io.y := Mux(io.sel, io.b, io.a)
}
```

Module 是模块的基础类型。IO 定义模块边界上的端口，Bundle 把多个端口组织成一组，Bool() 表示 1 位布尔类型，Input / Output 说明方向。

io.y 表示访问 io 里的 y。Mux(条件, 真时输入, 假时输入) 对应 Verilog 的三目运算符。因此最后一行是 sel=1 时输出 b，否则输出 a。

把两种写法对起来

Chisel	Verilog 中相近的描述
Input(Bool())	input wire a
Output(Bool())	output wire y
io.y := Mux(io.sel, io.b, io.a)	assign y = sel ? b : a
UInt(4.W)	4 位无符号向量

这里只比较行为与接口的对应关系，生成的 RTL 名字和排版未必与手写代码一致。优化工具也可能化简逻辑。

“写了什么”与“生成了什么”

写 Chisel 时，你在执行一个构造电路的 Scala 程序；生成结果则是硬件描述。运行构造程序时执行的 println，只是在电脑上打印文字。电路上的运行时打印需要专门的仿真支持，不能混为一谈。

练习：假设 a=1、b=0、sel=1，y 是多少？这里需要时钟吗？

2.3 寄存器与条件更新

将 RTL 计数器翻译成 Chisel

```scala
import chisel3._

class Counter4 extends Module {
  val io = IO(new Bundle {
    val enable = Input(Bool())
    val count = Output(UInt(4.W))
  })
  val count = RegInit(0.U(4.W))
  when(io.enable) {
    count := count + 1.U
  }
  io.count := count
}
```

RegInit 表示带复位初值的寄存器。标准 Module 提供时钟与复位接口。when 用来描述硬件条件；没有命中更新条件时，这个寄存器保持原值。[C4]

本例将加法结果接回 4 位寄存器，超出位宽的高位无法保留，所以会出现与 01 中相同的模 16 计数行为。若需要显式保留进位，必须设计更宽的输出或寄存器。

Scala if 与 Chisel when 的区别

Scala 的 if 通常在构造电路时决定生成哪种结构，例如是否生成一个调试模块；Chisel 的 when 由电路运行时的硬件信号决定某条连接如何选择。

```scala
val width = 4
val count = RegInit(0.U(width.W))
when(io.enable) { count := count + 1.U }
```

width 在生成时决定寄存器有几位；enable 在电路运行时决定是否更新。这两个“时间”应当分开理解。

同步复位的理解

教学对应图按普通同步复位理解，具体生成的复位类型以模块及生成结果为准。Chisel 还支持异步与抽象复位类型；本周只需知道复位语义应当明确，不能只看变量名猜测。[C3]

练习：这个模块生成完成后，修改 Scala 里的 width 会立刻改变已经配置好的 FPGA 吗？需要经过哪些步骤？

2.4 实验准备：固定一组教学依赖

在已有环境里做最小实验

配套 chisel_project.scala 固定使用 Scala 2.13.16 与 Chisel 7.2.0，目的是让示例可复现，不表示这些是最新版本。编译器插件版本与 Chisel 版本配套，Scala 完整版本也要匹配。

```scala
//> using scala "2.13.16"
//> using dep "org.chipsalliance::chisel:7.2.0"
//> using plugin "org.chipsalliance:::chisel-plugin:7.2.0"
//> using options "-language:reflectiveCalls"
```

这些以 //> 开头的行是 Scala CLI 指令，告诉工具下载什么依赖。普通 // 则是代码注释。第一次运行需要网络、下载依赖及 Java 环境，后续通常可利用缓存。

Windows：PowerShell

WinGet 可用时，安装 Scala CLI：

```powershell
winget install virtuslab.scalacli
```

macOS：终端

先按 05 安装 Homebrew，再安装 Scala CLI：

```bash
brew install scala-cli
```

Ubuntu / WSL：Ubuntu 终端

按 [C6] 的 Linux 安装步骤准备 Scala CLI，已有安装可直接使用。三个系统安装后均以 scala-cli version 检查，首次依赖下载需要网络。

同一组教学依赖

本套入口指定 --jvm 21，让 Scala CLI 管理 Java 21，减少跟随系统默认 JDK 变化引起的问题。当前终端中 java -version 不成功，也不必然说明 Scala CLI 管理的 JDK 不可用。

> macOS 用原生架构的工具，Windows 后端不支持时可转 WSL。各系统完整步骤和报错处理见 05，不需要同时安装 SBT、Mill 和完整 Chipyard。[C2][C6]

2.5 生成自己的安全门 RTL

完整模块已经放在配套文件里

SafetyGate.scala 中最核心的部分如下；完整文件还包含 import、输入输出定义和生成入口。

```scala
val stopped = RegInit(false.B)
when(io.killReq) {
  stopped := true.B
}
io.stopped := stopped
io.allow := io.request && io.authorized && !stopped
```

这段代码与 01 的停止门有相同的主要状态行为：复位清除停止位，采样到停止请求后保持停止，allow 由请求、授权和停止位共同决定。

在配套目录运行：

```bash
scala-cli run chisel_project.scala SafetyGate.scala --main-class EmitSafetyGate --jvm 21
```

上面的命令在三系统安装好工具后均适用；也可用 05 的分系统入口并把末尾参数改成 chisel。 入口中的 ChiselStage.emitSystemVerilog 把电路生成成 SystemVerilog 字符串，再打印到终端。示例关闭了后端随机初始化附加代码，便于用 Icarus 检查；寄存器仍应按规范复位。结果应包含 clock、reset、请求与授权端口、stopped 寄存器及 allow 逻辑。

不要只找“一模一样的一行代码”

后端可能重命名内部信号或把条件更新化简。比较时检查：输入输出方向、寄存器初值、何时更新、未更新时是否保持、组合输出规则。这些比格式更重要。

必做推演

先复位，再输入请求与授权，再采样 killReq=1，最后撤回 killReq。把每一步 stopped 和 allow 的值写下来，并与 01 的逐拍表对照。

生成与仿真是两个步骤

成功打印 RTL 后，还需测试平台检查行为。配套提供 SafetyGate.generated.sv，是从本例实际生成结果中提取的主模块；适配文件把端口接到 01 的同一套测试上。示例说明给出运行命令与验证范围。

2.6 常见报错与小练习

按报错发生的位置排查

现象	常见原因	先做什么
找不到 scala-cli	未安装或 PATH 未更新	重新开终端，执行 version
dependency 下载失败	网络、仓库访问或版本错误	核对固定依赖，保留完整报错
提示 compiler plugin	插件缺失或版本不配套	同时使用 chisel_project.scala
把 Bool 用到 Scala if 出错	混淆构造时参数与硬件条件	对硬件条件使用 when
输出未连接	没有给所有输出路径定义值	从模块接口逐个检查
firtool / 平台错误	RTL 生成后端不可用或不匹配	查官方安装与平台支持

练习 A：授权门

给一个已有模块的输出接上规则：“有请求、授权有效、且没有暂停”。只写最后一行 Chisel 连接，端口名为 request、authorized、paused、allow。

练习 B：故意找区别

比较下面两段：

```scala
val x = Wire(Bool())
x := io.killReq
```

```scala
val x = RegInit(false.B)
when(io.killReq) { x := true.B }
```

第一段与第二段是否都有记忆？killReq 撤回后分别会怎样？

练习 C：停止优先级

已有同步复位模块默认复位会清除停止位。如果研究要求“停止优先于恢复”，你应先修改什么规范、再检查哪些输入组合？允许写自然语言，不要求本周实现复杂复位结构。

2.7 参考答案与工具链复习

本册练习答案

width=8 是构造电路的参数；Wire(UInt(8.W)) 创建 8 位硬件信号。二选一选择器在 a=1、b=0、sel=1 时输出 0，不需要时钟。

修改生成器参数后，要重新运行生成、综合与实现，并重新配置 FPGA，才能改变对应硬件结构。具体还需看工程构建流程。

练习 A：

```scala
io.allow := io.request && io.authorized && !io.paused
```

练习 B：Wire 直接跟随 killReq，不保存历史；寄存器版本在采样到 1 后记住它，命令撤回也不会自动解除。

练习 C：先定义停止与恢复的优先级、谁有权恢复、复位阶段是否允许输出；再检查同时到来的停止和恢复、复位期间请求、停止命令撤回、连续重复停止等情况。不要仅靠名字叫 reset 就默认它可信。

本周对 Chipyard 的学习深度

知道 Chipyard 将处理器生成器、外围模块与验证工具组织成系统平台即可。先能独立读小模块、生成 RTL、解释仿真，再研究将安全模块接入 SoC 的哪条路径。完整搭建 Chipyard 可留待平台与架构方向明确之后。[C5]

阅读来源

[C1] Chisel 官方介绍：https://www.chisel-lang.org/docs

[C2] Chisel 安装与后端说明：https://www.chisel-lang.org/docs/installation

[C3] 复位类型：https://www.chisel-lang.org/docs/explanations/reset

[C4] 时序电路：https://www.chisel-lang.org/docs/explanations/sequential-circuits

[C5] Chipyard 中的 Chisel：https://chipyard.readthedocs.io/en/stable/Tools/Chisel.html

[C6] Scala CLI 安装：https://scala-cli.virtuslab.org/install/

官方文档会更新。这里固定的教学版本用于示例复现；后续工程选型应核对导师平台与依赖兼容性。

制作验证：本例已在 Ubuntu / WSL 实际生成 RTL，并用 Icarus 通过与手写停止门相同的定向测试；未进行 FPGA 验证。详细环境见配套验证记录。

语法补充 C1：Scala 外壳与硬件类型

必学：同一份文件里有两种层次

```scala
import chisel3._
import chisel3.util._

class Pass8 extends Module {
  val io = IO(new Bundle {
    val in = Input(UInt(8.W))
    val out = Output(UInt(8.W))
  })
  io.out := io.in
}
```

这是完整模块定义，生成入口和依赖需要另行配置。import 引入库中名字，util 包含 Cat、switch 等工具。class 定义类型，extends Module 表示这是 Chisel 模块；花括号组织代码块。多数语句不必写分号。

从内向外读一个端口

UInt(8.W) 是“8 位无符号硬件类型”；Input 表示方向；Bundle 将端口打包；IO 将其定义为模块接口；val io = ... 给接口对象起名。访问里面的端口写 io.in 或 io.out。[C7]

常量与类型速查

写法	含义	与 Verilog 对照
Bool()	1 位布尔硬件类型	单 bit 条件信号
UInt(8.W)	8 位无符号硬件类型	8 位无符号向量
SInt(8.W)	8 位有符号硬件类型	signed 向量
12.U(8.W)	8 位无符号常量 12	8'd12
"h0C".U(8.W)	十六进制常量	8'h0C
"b1100".U(8.W)	二进制常量	8'b00001100
true.B、false.B	硬件布尔常量	1'b1、1'b0

val width = 8 中的 8 是 Scala 整数，常用于决定生成结构；8.U 则是硬件数值。// 与 /* ... */ 都可写注释；名字区分大小写。不要把 val 理解成“硬件值永远不变”。

语法补充 C2：连接、比较与位运算

必学：=、:=、=== 分别做什么

写法	实际含义	例子
val x = ...	Scala 建立名字与对象的绑定	val x = Wire(UInt(8.W))
x := y	描述硬件连接或更新输入	io.out := io.in
x === y	硬件相等比较，结果是 Bool	io.a === io.b
x =/= y	硬件不等比较	io.a =/= 0.U
x <= y	硬件小于等于比较	io.count <= 10.U

Chisel 的 <= 不承担 Verilog 非阻塞赋值的作用。寄存器用 := 描述更新关系；保存状态的能力来自 Reg / RegInit，而不是从某一种赋值符号自动推断。[C8]

```scala
val same = io.a === io.b
val allowed = io.request && !io.stopped
val bits = io.a & io.b
val inverse = ~io.a
val low = io.a(3, 0)
val first = io.a(0)
val joined = Cat(io.a(3, 0), io.b(3, 0))
```

此处是假定端口已声明的片段。&&、||、! 用于 Bool 的逻辑运算；&、|、^、~ 用于按位运算。切片用圆括号，单 bit 用 a(0)；Cat 的第一个参数放在高位。

两个容易看错的地方

在 Chisel 中用 === 比较硬件，不能照抄 Verilog 的 ==。而 Verilog 里的 === 常用于包含 X/Z 的严格仿真比较；不要把两门语言相同符号的所有语义画等号。

x.asUInt 是将位按无符号解释，x.asSInt 是按有符号解释，不会凭空增加精度。例如 8 位全 1 按 UInt 为 255，按二进制补码 SInt 为 -1。

复杂条件用括号，例如 (io.a === io.b) && io.enable。若看到类型错误，先看你把 Scala Boolean 与硬件 Bool 混在一起没有。

语法补充 C3：Wire、Reg 与分支

必学：让不同构件承担不同职责

构件	记忆能力	本周怎样用
Wire(UInt(8.W))	无	稍后连接的组合中间信号
WireDefault(0.U(8.W))	无	带默认连接的组合信号
Reg(UInt(8.W))	有	寄存器，但这里未规定复位值
RegInit(0.U(8.W))	有	复位时回到 0 的寄存器
RegNext(io.in)	有	把输入延迟一个采样周期

WireDefault 的“默认”是电路连接的默认选择，不是“开机只赋一次值”。组合信号需要完整连接；寄存器可在某些条件下不更新，保持原值。[C4]

```scala
val y = WireDefault(0.U(8.W))
when(io.enable) {
  y := io.a
}.elsewhen(io.fallback) {
  y := io.b
}.otherwise {
  y := 0.U
}
io.y := y
```

when / elsewhen / otherwise 依次描述条件选择，前面的条件优先。上例在 enable 与 fallback 同为真时选择 a；如果只是二选一，可用 io.y := Mux(io.sel, io.b, io.a)。[C10]

```scala
val count = RegInit(0.U(4.W))
when(io.enable) { count := count + 1.U }
```

这是状态更新片段，Module 提供相应时钟与复位。读右侧 count 时取当前状态，写左侧时描述寄存器下一状态的输入，不会形成“自己加自己直到无穷”的组合循环。

Scala if 与 Chisel when

if (width == 8) 由生成时的 Scala 参数决定要构造什么；when(io.enable) 由硬件运行时输入决定电路选择。不能把 Bool 直接放进 Scala if 充当 Boolean。构造时与电路运行时应始终分开。

语法补充 C4：模块实例、参数与数组

必学：把小电路接到大电路

```scala
val gate = Module(new SafetyGate)
gate.io.request := io.request
gate.io.authorized := io.authorized
gate.io.killReq := io.killReq
io.allow := gate.io.allow
```

假定这段放在父模块内，且父接口已声明。Module(new ...) 实例化已有模块；gate 是实例名。子模块输入在左边接父输入，父输出在左边接子输出。常规 Module 子模块还会继承相应的时钟和复位上下文。[C9]

Scala 构造参数决定结构

```scala
class PassN(width: Int) extends Module {
  require(width > 0)
  val io = IO(new Bundle {
    val in = Input(UInt(width.W))
    val out = Output(UInt(width.W))
  })
  io.out := io.in
}
```

实例化时 Module(new PassN(8)) 生成 8 位版本。require 在生成时检查参数。width 是 Scala Int，没有形成一个运行中能改变宽度的输入端口。

选读：Vec 与生成循环

```scala
val lanes = Wire(Vec(4, UInt(8.W)))
for (i <- 0 until 4) {
  lanes(i) := i.U(8.W)
}
io.out := lanes(io.index)
```

假定 io.index 为 2 位 UInt，io.out 为 8 位 UInt。Vec 包含四个 8 位元素；until 4 不包括 4。循环生成四项连接，动态索引则形成选择逻辑，不代表逐个时钟遍历。[C11]

Vec(4, UInt(8.W)) 是四项 8 位数据；UInt(4.W) 是一个 4 位数。涉及存储容量更大或多端口时再学习 Mem 等结构，本周先能区分这两种表达。

语法补充 C5：switch 与加法位宽

必学：直接对照 Verilog case

```scala
val y = WireDefault(0.U(8.W))
switch(io.op) {
  is(0.U) { y := io.a + io.b }
  is(1.U) { y := io.a & io.b }
  is(2.U) { y := Cat(io.a(3, 0), io.b(3, 0)) }
}
io.y := y
```

假定 a、b 和 y 为 8 位、op 为 2 位，已经 import chisel3.util._。没有匹配 is 时，y 使用之前给定的默认连接；这里是 0。可将它与 V6 中 case 的四种输出一一核对。

+ 与 +& 的区别

```scala
val truncated = io.a + io.b
val full = io.a +& io.b
```

当两个输入都是 8 位 UInt，第一种加法结果为 8 位，第二种扩展为 9 位。a=240、b=60 时，分别得到 44 和 300。即使把已经截断的结果接到 9 位输出，也不能找回丢掉的进位。[C12]

因此完整示例使用：

```scala
io.sumFull := io.a +& io.b
```

sumFull 的端口也声明为 9 位。端口、计算与接收寄存器三处位宽都要检查。初学时把端口和状态位宽写清楚，其他推断等熟练后再用。

常见错误的改法

容易写错	应先这样检查
用 a == b 比较硬件	改为 a === b，结果应为 Bool
用 Scala if 判断 io.enable	改为硬件 when 或 Mux
Wire 没有覆盖所有路径	提供默认连接或完整分支
只扩输出端口，不扩加法	需要进位时用 +& 并检查目标宽度
改 width 后期待旧 FPGA 自动变化	需要重新生成与实现

语法补充 C6：运行与 Verilog 同功能的电路

动手：先对照功能，再对照文本

配套 SyntaxBasics.scala 的完整接口与 V6 相同，主要逻辑如下。构建入口名是 EmitSyntaxBasics；不要误用前面安全门的入口名。

完整文件的 _root_.circt... 表示从根包查找生成工具，避免与 util 中同名包混淆；先知道用途，不必背这段导入路径。

```scala
val y = WireDefault(0.U(8.W))
switch(io.op) {
  is(0.U) { y := io.a + io.b }
  is(1.U) { y := io.a & io.b }
  is(2.U) { y := Cat(io.a(3, 0), io.b(3, 0)) }
}
io.y := y
io.sumFull := io.a +& io.b
val count = RegInit(0.U(4.W))
when(io.enable) { count := count + 1.U }
io.count := count
```

第一步：生成 RTL

VS Code“终端 → 运行任务 → 语法：Chisel 综合示例生成 RTL”。任务会选择本机已有的 Java 21；也可在配套目录直接执行：

```bash
scala-cli run chisel_project.scala SyntaxBasics.scala \
  --main-class EmitSyntaxBasics --server=false --jvm 21
```

此前 JDK 下载若卡住，仍可用 05 介绍的 --java-home 指向已有 Java 21。首次运行库和后端需要网络。

第二步：仿真已附带的生成模块

```bash
python3 run_examples.py rtl --example syntax_chisel
```

它使用 SyntaxBasics.generated.sv 和端口适配器，复用 V6 的同一仿真文件。这个命令检查仓库已有的生成结果，不自动把刚打印的多文件文本覆盖进去。

两种实现应满足相同输入输出与计数规则。生成工具可能化简内部逻辑，不能要求生成文本与手写 Verilog 完全一样。当前实验验证指定功能与输入集合，不等于完整系统安全证明。

语法补充 C7：两种语言对照与自测

先用已知 Verilog 读懂 Chisel

要表达什么	Verilog / SystemVerilog	Chisel
8 位输入	input wire [7:0] a	val a = Input(UInt(8.W))
8 位常量 12	8'd12	12.U(8.W)
组合选择	sel ? b : a	Mux(sel, b, a)
低四位	a[3:0]	a(3, 0)
拼接	{a,b}	Cat(a,b)
普通硬件相等判断	a == b	a === b
带复位寄存器	时钟 always、复位分支、非阻塞赋值	RegInit 与 when、:=
组合默认值	always 块开头赋默认值	WireDefault 或显式默认连接

这些是入门对应关系，不是逐字符翻译规则。特别是 Chisel := 接在 Wire 还是 Reg 上，决定它描述的是组合连接还是寄存器更新输入。

自测与答案

1. val 会阻止寄存器更新吗？不会，它固定 Scala 名字的绑定。
2. 已丢进位的结果接到宽输出能恢复吗？不能，应使用 +&。
3. a=8'hF0、b=8'h3C、op=2，y 是多少？8'h0C，取低四位拼接。

官方语法入口

[C7] 数据类型：https://www.chisel-lang.org/docs/explanations/data-types

[C8] 运算符：https://www.chisel-lang.org/docs/explanations/operators

[C9] 模块：https://www.chisel-lang.org/docs/explanations/modules

[C10] 选择器：https://www.chisel-lang.org/docs/explanations/muxes-and-input-selection

[C11] 数组：https://www.chisel-lang.org/docs/explanations/bundles-and-vecs

[C12] 位宽：https://www.chisel-lang.org/docs/explanations/width-inference